NimbusPay: learning ticket-triage rules

Final result: raw-data 16-bit LoRA achieves 71.50% exact match (143/200) and 94.56% mean field accuracy. All 400 test outputs were generated. A fresh T4 reproduction took 60.13 pipeline minutes; the executed notebook reports 62.26 minutes including setup/export.

1. Data audit and decisions

The audit reads only train.jsonl and implements rules from SCHEMA.md. Of 1,445 rows, 60 exact duplicate extras and 25 invalid/truncated targets were dropped. Conflicting exact-input groups: zero. The cleaned set contains 1,360 rows, with 218 retained rows modified. Each drop/fix is recorded once in cleaning_log.csv; reasons may overlap.

Evidence methods: parse each target as JSON; compare key sets/types; group identical ticket strings; normalize NP identifiers; check category-specific null fields, explicit claim amounts, date formats and priority precedence. Findings include 70 legacy prio records, 30 legacy slash-date repairs, 20 identifier repairs, 21 explicit-amount corrections and 62 priority corrections. The cleaner removes quoted email lines from 48 retained inputs. These counts are overlapping operations, not separate row totals.

Detection also found 40 inputs longer than 1,000 characters and 77 duplicate extras after whitespace normalization (including the 60 exact duplicates). Non-exact near duplicates were retained conservatively. Invalid targets were dropped rather than reconstructed. Unknown semantic category/language errors, unhandled date formats and ambiguous claims may remain. No model/API generated replacement labels; deterministic rules do the repairs. Audit before/after evidence can be regenerated locally without publishing training tickets.

2. Training and inference

Base: Qwen/Qwen2.5-0.5B-Instruct, revision 7ae557604adf67be50417f59c2c2f167def9a775. Hugging Face Transformers + PEFT keeps the native chat template and standard adapter format explicit. A 0.5B model permits several T4 experiments, at the cost of limited arithmetic/rule capacity. Final LoRA uses rank 16, alpha 32, dropout 0.05, attention plus MLP projections and 8,798,208 trainable parameters, below the 50M cap. Base weights remain frozen; final loading is FP16, not 4-bit.

Two epochs, learning rate 2e-4, cosine decay, 5% warmup, weight decay 0.01, microbatch 1 and accumulation 16 are shared starting settings. Gradient checkpointing limits memory. Loss masks all prompt and padding tokens (-100), retaining assistant response and end marker. Prefix equality is asserted against the generation template.

Raw token lengths: median 168, p95 225, p99 1,804.12 and maximum 2,215; recorded padded ceiling 2,240, with zero truncation. Clean runs use ceiling 288. Inference uses untouched system/user messages, greedy model.generate, max_new_tokens=200 and no rule prompt, constrained decoding, fallback or field repair. Rules are appended only in the separate reference baseline.

Controlled experiments

All dev measurements use the supplied score.py. Rows below are real completed runs, each using seed 42. GPU values are peak allocated GiB measured from model loading through training; reserved memory is saved separately. Training minutes exclude download, generation and scoring.

Run	Dev EM	Field acc.	Peak GiB	Train min
Base: fixed	0.00%	0.00%	-	-
Base: rules	0.00%	2.31%	-	-
clean_fp16	66.00%	93.94%	1.56	18.62
raw_fp16	71.50%	94.56%	4.94	20.42
clean_qlora	63.50%	93.38%	1.58	26.10
rank32_fp16	63.00%	93.31%	1.69	18.83
compact_fp16	62.50%	93.25%	1.53	18.75

Fixed-prompt baseline: 1.0% JSON-object validity. Rules-prompt baseline: 6.5% validity. Both have zero exact matches under the strict scorer and 200-token cap. The saved scores establish formatting failure, but raw baseline text is not in this export, so a more specific explanation would be speculative.

Four comparisons and interpretation

Cleaning: raw vs clean changes the complete cleaning policy. Clean loses 5.5 percentage points of exact match, while reducing training memory from 4.94 to 1.56 GiB. Dropped examples, removal of quoted chains and heuristic corrections may each contribute; this bundled experiment cannot isolate a cause. The selected final model therefore uses all 1,445 raw rows, including known noisy labels.

Quantization: clean FP16 vs NF4 QLoRA changes base loading and required quantization preparation. QLoRA loses 2.5 points, takes 26.10 versus 18.62 minutes, and shows no peak allocated-memory saving here (1.58 versus 1.56 GiB). For this small model, quantization overhead and preparation can outweigh base-weight savings; this is a hypothesis, not a measured memory decomposition.

Rank: 16 vs 32 loses 3 points and doubles trainable parameters to 17,596,416. Alpha stays 32, so alpha/r changes alongside capacity; this is a limitation of the rank comparison. More trainable parameters did not improve this single-seed result.

Own idea: compact JSON training targets remove serialization whitespace, with other clean-run settings held fixed. Exact match drops 3.5 points to 62.5%. This target-format change offered no observed accuracy advantage.

All experiments hold epochs fixed, so raw and cleaned sets have different optimizer-step counts (180 versus 170). Their input lengths also differ. There are no repeated seeds or significance claims. Dev chooses the configuration; test has no labels and is never used for training or selection.

Reproduction and scope

The final fresh run reproduces 71.50% / 94.56%, uses 4.94 GiB peak allocated memory and takes 19.98 training minutes. The saved adapter is reloaded before dev evaluation; a five-row greedy repeat check succeeds after generating the test file. Evidence records resumed_or_reused=false. Weights and test output bytes are unchanged during packaging. Portable notebook setup/summary cells were edited afterward; the original executed notebook is retained as evidence. No new GPU reload was performed during packaging.

Error analysis and next steps

All 200 final dev outputs parse as JSON objects, yet 57 tickets fail exact match. Field errors: priority 48, needs_human 20, category 8, date 7, amount 3, channel 1, ID 0 and language 0 (87 errors total). Priority errors occur in 48 of the 57 failed tickets. Weakest category exact matches: refund 52.73% (55 rows), payment_failed 58.49% (53), fraud 66.67% (15). These are small, unequal groups.

Ten specific failures below show predicted values versus required values. Explanations describe observable rule violations; suggested mechanisms are hypotheses. Full input, gold record and raw output are preserved in failures.jsonl.

Dev ticket	Observed failure and explanation
dv-00005	Rs 49,680 is below Rs 50,000: P1/true should be P2/false. Amount extraction is correct; threshold application is wrong.
dv-00006	Received July 2; day before yesterday is June 30, not July 1. The relative-date offset is one day short across a month boundary.
dv-00011	The Rs 84,137 refund requires P1/true, but output is P3/false despite the correct amount. The high-value escalation rule is missed.
dv-00016	The Rs 25,647 failed top-up requires P2, not P3. URGENT alone is not a P1 trigger; the amount threshold is sufficient for P2.
dv-00018	The misspelled Hinglish text describes a cancelled order awaiting refund. Output payment_failed should be refund; other fields match.
dv-00023	Output P3 and 14/09/2026 should be P2 and 2026-09-14. The amount threshold and ISO date format are both violated.
dv-00037	Output 8,446,900 paise should be 28,466,900 for Rs 284,669. Even the predicted amount exceeds the P1 threshold, yet output is P3/false.
dv-00042	Card-delivery tracking is other, not refund. The card mention is not a payment mode; channel must be null for this category.
dv-00067	An unrecognized wallet charge is fraud, not refund. This intent error also changes required P1/true to P3/false.
dv-00164	The customer explicitly says again and again about PIN unblocking. needs_human is false instead of true; the long quoted chain may distract, but causation is unproven.

One more week

First manually audit ambiguous training labels and isolate cleaning components while preserving long quoted-email examples. Add deterministic train-derived boundary examples around Rs 5,000/Rs 50,000, month/year transitions and unauthorized transactions; never create them from dev/test rows. Then compare label-only cleaning, repeated seeds and a larger eligible model under the same T4 budget. Check whether gains survive strict greedy decoding and the 200-token cap.

AI assistance and evidence

ChatGPT/Codex assisted with deterministic audit/training code, debugging, tests, packaging and post-run error explanations. The candidate executed the Colab experiments. No external AI model/API generated target labels or test predictions. Report values come from saved scorer metrics and original notebook outputs; no results were fabricated. Source, configuration, environment and logs are included. Some text derives from PolyAI Banking77 (CC-BY-4.0), per the assignment.